

MovieHub

Dokumentacja Techniczna

Aplikacja desktopowa do oceniania filmów

Autor	Kamil Kozicz
Uczelnia	Collegium Da Vinci
Numer indeksu	31828
Rok akademicki	2025/2026

ASP.NET Core 10 · Entity Framework Core · PostgreSQL 16 · Avalonia UI · MVVM · Azure

Spis treści

1. Model relacyjny bazy danych.....	5
1.1. Encja User — użytkownik systemu.....	5
1.2. Encja Genre — gatunek filmowy.....	5
1.3. Encja Director — reżyser.....	6
1.4. Encja Movie — film.....	6
1.5. Encja Rating — ocena.....	7
1.6. Diagram relacji encji (ERD).....	8
2. Warstwa dostępu do danych — ApplicationDbContext.....	9
2.1. Właściwości DbSet.....	9
2.2. Konfiguracja Fluent API.....	9
3. Wzorzec Data Transfer Objects (DTO).....	9
3.1. Uzasadnienie zastosowania wzorca DTO.....	9
3.2. DTOs warstwy autentykacji.....	10
RegisterRequest.....	10
LoginRequest.....	10
AuthResponse.....	10
3.3. DTOs warstwy filmów.....	10
CreateMovieRequest / UpdateMovieRequest.....	10
MovieSummaryResponse.....	10
MovieDetailResponse.....	11
MovieListQueryRequest.....	11
3.4. DTOs warstwy ocen.....	11
CreateRatingRequest / UpdateRatingRequest.....	11
RatingResponse.....	11
3.5. DTOs warstwy reżyserów i gatunków.....	11
3.6. Obiekty podsumowujące (Summary DTOs).....	11
4. Architektura obsługi błędów — wyjątki domenowe.....	12
4.1. Filozofia projektowania.....	12
4.2. Katalog wyjątków i ich mapowanie na kody HTTP.....	12
NotFoundException.....	12
ConflictException.....	12
DuplicateUserException.....	12
InvalidCredentialsException.....	13
ForbiddenException.....	13
4.3. Tabela mapowania wyjątków na kody HTTP.....	13
4.4. Integracja z pipeline'em ASP.NET Core.....	13
5. Serwisy — warstwa logiki biznesowej.....	14
5.1. AuthService.....	14
RegisterAsync(RegisterRequest request, CancellationToken cancellationToken).....	14
LoginAsync(LoginRequest request, CancellationToken cancellationToken).....	14
GenerateJwtToken(User user) — metoda prywatna.....	14
5.2. MovieService.....	15
GetAllAsync(MovieListQueryRequest query, CancellationToken cancellationToken).....	15
GetByIdAsync(int id, CancellationToken cancellationToken).....	15

CreateAsync(CreateMovieRequest request, CancellationToken cancellationToken).....	15
UpdateAsync(int id, UpdateMovieRequest request, CancellationToken cancellationToken)...	15
DeleteAsync(int id, CancellationToken cancellationToken).....	16
5.3. DirectorService.....	16
5.4. GenreService.....	16
5.5. RatingService.....	16
CreateAsync(int movieId, int userId, CreateRatingRequest request, CancellationToken cancellationToken).....	16
UpdateAsync(int userId, int ratingId, UpdateRatingRequest request, CancellationToken cancellationToken).....	17
6. Kontrolery — warstwa interfejsu API.....	17
6.1. AuthController — trasa bazowa: /api/auth.....	17
POST /api/auth/register.....	17
POST /api/auth/login.....	18
6.2. MoviesController — trasa bazowa: /api/movies.....	18
GET /api/movies.....	18
GET /api/movies/{id}.....	18
POST /api/movies — [Authorize(Roles = "Admin")].....	19
PUT /api/movies/{id} — [Authorize(Roles = "Admin")].....	19
DELETE /api/movies/{id} — [Authorize(Roles = "Admin")].....	19
6.3. DirectorsController — trasa bazowa: /api/directors.....	19
GET /api/directors.....	20
POST /api/directors — [Authorize(Roles = "Admin")].....	20
6.4. GenresController — trasa bazowa: /api/genres.....	20
GET /api/genres.....	20
POST /api/genres — [Authorize(Roles = "Admin")].....	21
6.5. RatingsController — trasa bazowa: /api.....	21
POST /api/movies/{movieId}/ratings — [Authorize].....	21
PUT /api/ratings/{ratingId} — [Authorize].....	22
6.6. Pełna tabela endpointów API.....	22
7. Architektura frontendu — Avalonia MVVM.....	23
7.1. Cykl życia i proces uruchamiania aplikacji.....	23
Klasa Program — punkt wejścia i konfiguracja platformy.....	23
Klasa App — rejestracja zależności i inicjalizacja głównego okna.....	24
7.2. Mechanizm ViewLocator — automatyczne wiązanie widoków z modelami widoków.....	24
Rola interfejsu IDataTemplate.....	24
Konwencja nazewnictwa jako mechanizm wiązania.....	24
7.3. CommunityToolkit.Mvvm — mechanizm generowania kodu.....	25
[ObservableProperty].....	25
[NotifyCanExecuteChangedFor].....	25
[RelayCommand].....	26
Klasa bazowa ViewModelBase.....	26
7.4. Serwisy infrastrukturalne.....	26
SessionService.....	26
NavigationService.....	27
ApiClient.....	27

7.5. Lokalna obsługa błędów — PasswordMissMatchException.....	27
7.6. Diagram zależności infrastruktury klienta.....	28
8. Moduły prezentacji — ViewModele i widoki.....	28
8.1. Moduł Auth — logowanie i rejestracja.....	28
AuthViewModel.....	28
AuthView.axaml — struktura widoku.....	29
8.2. Moduł Browse — przeglądanie katalogu filmów.....	30
BrowseViewModel.....	30
MovieSummaryViewModel.....	30
BrowseView.axaml — struktura widoku.....	31
8.3. Moduł MovieDetail — szczegóły i ocenianie.....	31
MovieDetailViewModel.....	31
MovieDetailView.axaml — struktura widoku.....	32
8.4. Moduł Admin — panel zarządzania katalogiem.....	32
Architektura panelu Admin.....	32
AdminViewModel i sub-ViewModele formularzy.....	33
AdminView.axaml — struktura widoku.....	33
8.5. Podsumowanie przepływu danych.....	33
8.6. Kluczowe właściwości architektoniczne.....	34
9. Wdrożenie produkcyjne — Azure.....	34
9.1. Zasoby Azure.....	34
9.2. CI/CD — GitHub Actions.....	35
9.3. Zmienne środowiskowe (App Service).....	35
9.4. Dostęp do bazy danych i zarządzanie.....	35
10. Struktura projektu.....	36
10.1. Struktura katalogów.....	36
10.2. Konwencje nazewnictwa i organizacja kodu.....	36
11. Podsumowanie.....	37
11.1. Zrealizowane wymagania funkcjonalne.....	37
11.2. Zastosowane technologie — zestawienie.....	37

1. Model relacyjny bazy danych

Projekt MovieHub opiera się na relacyjnym modelu danych zaimplementowanym przy użyciu biblioteki Entity Framework Core w podejściu Code-First. Model domenowy składa się z pięciu encji: User, Movie, Rating, Genre oraz Director, które tworzą spójną sieć powiązań odzwierciedlającą rzeczywistą strukturę systemu katalogowania filmów z funkcją oceniania.

1.1. Encja User — użytkownik systemu

Klasa User reprezentuje zarejestrowanego użytkownika platformy. Posiada następujące właściwości:

Właściwość	Typ	Ograniczenia
------------	-----	--------------

Id	int	Klucz główny, generowany automatycznie
Username	string	Wymagany, maks. 50 znaków, unikalny indeks
Email	string	Wymagany, maks. 100 znaków, unikalny indeks
PasswordHash	string	Wymagany, przechowuje skrót hasła (BCrypt, koszt 12)
Role	UserRole	Wyliczenie: User (0) lub Admin (1)
RegisteredAt	DateTime	Znacznik czasu rejestracji (UTC)
Ratings	ICollection<Rating>	Nawigacyjna kolekcja ocen wystawionych przez użytkownika

Encja User pozostaje w relacji jeden-do-wielu z encją Rating: jeden użytkownik może wystawić wiele ocen, lecz każda ocena należy do dokładnie jednego użytkownika. Unikalność pól Username oraz Email jest egzekwowana na poziomie bazy danych poprzez indeksy unikalne, co zapobiega duplikacji kont. Pole PasswordHash przechowuje wyłącznie kryptograficzny skrót hasła, nigdy jego wartość jawną, co stanowi fundamentalny wymóg bezpieczeństwa. Wyliczenie UserRole implementuje prosty model kontroli dostępu oparty na rolach (RBAC), rozróżniając zwykłych użytkowników (User) od administratorów (Admin).

1.2. Encja Genre — gatunek filmowy

Klasa Genre reprezentuje kategorię gatunkową, do której może należeć film. Jej struktura jest celowo minimalistyczna:

Właściwość	Typ	Ograniczenia
Id	int	Klucz główny
Name	string	Wymagany, maks. 50 znaków, unikalny indeks
Movies	ICollection<Movie>	Nawigacyjna kolekcja filmów danego gatunku

Encja Genre pozostaje w relacji jeden-do-wielu z encją Movie: jeden gatunek może obejmować wiele filmów, natomiast każdy film przynależy do dokładnie jednego gatunku. Unikalność nazwy gatunku jest gwarantowana przez indeks unikalny na kolumnie Name, co eliminuje możliwość wprowadzenia zduplikowanych kategorii.

1.3. Encja Director — reżyser

Klasa Director modeluje osobę reżysera i zawiera podstawowe dane biograficzne:

Właściwość	Typ	Ograniczenia
Id	int	Klucz główny
FirstName	string	Wymagany, maks. 50 znaków
LastName	string	Wymagany, maks. 50 znaków
Nationality	string	Wymagana, maks. 50 znaków

Movies	ICollection<Movie>	Nawigacyjna kolekcja filmów danego reżysera
--------	--------------------	---

Encja Director pozostaje w relacji jeden-do-wielu z encją Movie: jeden reżyser może być twórcą wielu filmów, lecz każdy film posiada dokładnie jednego reżysera. Separacja encji reżysera od encji filmu umożliwia wielokrotne wykorzystanie danych reżysera bez ich redundancji, a także pozwala na filtrowanie filmografii konkretnego twórcy.

1.4. Encja Movie — film

Klasa Movie stanowi centralną encję modelu danych, agregującą informacje o filmie oraz powiązania z gatunkiem, reżyserem i ocenami:

Właściwość	Typ	Ograniczenia
Id	int	Klucz główny
Title	string	Wymagany, maks. 150 znaków
Year	int	Rok produkcji
Description	string	Wymagany, maks. 1000 znaków
PosterUrl	string?	Opcjonalny adres URL plakatu
GenreId	int	Klucz obcy do Genre
DirectorId	int	Klucz obcy do Director
Genre	Genre	Właściwość nawigacyjna do gatunku (non-nullable: null!)
Director	Director	Właściwość nawigacyjna do reżysera (non-nullable: null!)
Ratings	ICollection<Rating>	Nawigacyjna kolekcja ocen

Encja Movie uczestniczy w trzech relacjach: wiele-do-jednego z Genre (poprzez klucz obcy GenreId), wiele-do-jednego z Director (poprzez klucz obcy DirectorId) oraz jeden-do-wielu z Rating. Właściwości nawigacyjne Genre i Director są oznaczone jako non-nullable (null!), co wyraża semantyczne założenie, że film zawsze musi posiadać przypisany gatunek i reżysera.

1.5. Encja Rating — ocena

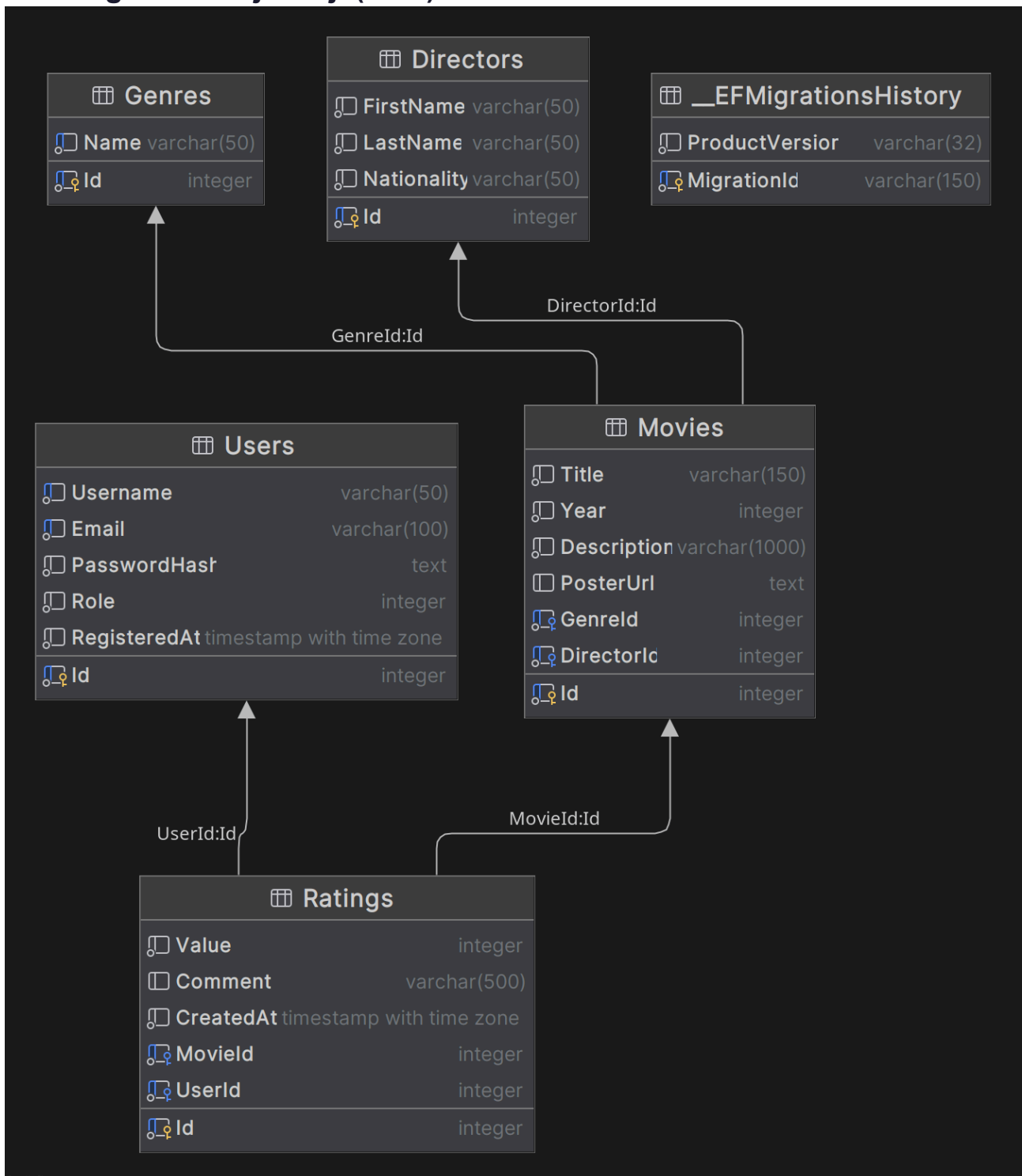
Klasa Rating modeluje ocenę wystawioną przez użytkownika konkretnemu filmowi:

Właściwość	Typ	Ograniczenia
Id	int	Klucz główny
Value	int	Wartość liczbowa oceny (1–10)
Comment	string?	Opcjonalny komentarz, maks. 500 znaków
CreatedAt	DateTime	Data wystawienia oceny (UTC)

MovieId	int	Klucz obcy do Movie
UserId	int	Klucz obcy do User
Movie	Movie	Właściwość nawigacyjna do filmu
User	User	Właściwość nawigacyjna do użytkownika

Kluczowym ograniczeniem integralności danych jest złożony indeks unikalny na parze kolumn (MovieId, UserId), który gwarantuje, że dany użytkownik może wystawić co najwyżej jedną ocenę danemu filmowi. Encja Rating pełni rolę tabeli łączącej w relacji wiele-do-wielu pomiędzy User a Movie, wzbogaconej jednak o dodatkowe atrybuty (wartość oceny, komentarz, data), co czyni ją pełnoprawną encją asocjacyjną, a nie jedynie tabelą pośrednią.

1.6. Diagram relacji encji (ERD)



Podsumowując strukturę relacyjną: Genre i Director są encjami słownikowymi (referencyjnymi), Movie jest encją centralną, natomiast Rating jest encją asocjacyjną łączącą Movie z User z dodatkowym ograniczeniem unikalności pary (MovieId, UserId).

2. Warstwa dostępu do danych — ApplicationDbContext

Klasa ApplicationDbContext dziedziczy po DbContext z biblioteki Entity Framework Core i stanowi główny punkt wejścia do bazy danych PostgreSQL (co potwierdza obecność pakietu Npgsql.EntityFrameworkCore.PostgreSQL w projekcie). Konstruktor klasy wykorzystuje wzorzec primary constructor języka C# 12, przyjmując obiekt DbContextOptions<ApplicationDbContext> i przekazując go do konstruktora klasy bazowej.

2.1. Właściwości DbSet

Klasa eksponuje pięć właściwości DbSet<T>, odpowiadających pięciu encjom modelu:

```
public DbSet<User>      Users      => Set<User>();
public DbSet<Genre>     Genres     => Set<Genre>();
public DbSet<Director> Directors => Set<Director>();
public DbSet<Movie>     Movies     => Set<Movie>();
public DbSet<Rating>    Ratings    => Set<Rating>();
```

2.2. Konfiguracja Fluent API

Metoda OnModelCreating zawiera konfigurację Fluent API, która precyzuje ograniczenia i relacje niemożliwe do wyrażenia wyłącznie przez konwencje EF Core. Konfiguracja ta obejmuje:

- Definicję indeksów unikalnych (dla User.Username, User.Email, Genre.Name oraz złożonego Rating.(MovieId, UserId))
- Ograniczenia długości pól tekstowych (HasMaxLength)
- Jawną deklarację relacji jeden-do-wielu przy użyciu metod HasOne, WithMany i HasForeignKey

Zastosowanie Fluent API zamiast atrybutów danych (Data Annotations) zapewnia wyraźne oddzielenie konfiguracji persystencji od klas domenowych, co jest zgodne z zasadą pojedynczej odpowiedzialności (SRP).

3. Wzorzec Data Transfer Objects (DTO)

3.1. Uzasadnienie zastosowania wzorca DTO

Wzorzec Data Transfer Object (DTO) został zastosowany w projekcie MovieHub w celu realizacji fundamentalnej zasady separacji warstw architektury. Bezpośrednie eksponowanie encji domenowych (klas z folderu Models) w interfejsie API niosłoby ze sobą szereg poważnych konsekwencji:

- Ujawnienie wrażliwych pól (takich jak PasswordHash w encji User)
- Nadmiarowe przesyłanie danych (over-fetching)
- Ryzyko cyklicznych referencji podczas serializacji JSON (np. Movie → Ratings → Movie)
- Ścisłe sprzężenie kontraktu API ze schematem bazy danych, uniemożliwiające niezależną ewolucję obu warstw

Obiekty DTO pełnią rolę precyzyjnie zdefiniowanych kontraktów pomiędzy warstwą prezentacji (kontrolerami API) a warstwą domenową, umożliwiając selektywne ujawnianie i przyjmowanie wyłącznie tych danych, które są niezbędne w danym kontekście operacji.

3.2. DTOs warstwy autentykacji

RegisterRequest

Obiekt żądania rejestracji nowego użytkownika. Przyjmuje trzy pola: Username, Email oraz Password. Kluczową właściwością tego DTO jest to, że pole Password zawiera hasło w postaci jawnej, które warstwa serwisowa zobowiązana jest przetworzyć (zahaszować) przed zapisem do bazy danych. Dzięki temu encja User nigdy nie przechowuje jawnego hasła, a DTO stanowi barierę ochronną uniemożliwiającą jego przypadkowe utrwalenie.

LoginRequest

Obiekt żądania logowania. Zawiera wyłącznie pola Email i Password, co stanowi minimalny zestaw danych wymaganych do uwierzytelnienia. Celowe pominięcie pola Username wymusza jednoznaczną identyfikację użytkownika przez adres e-mail.

AuthResponse

Obiekt odpowiedzi po pomyślnym uwierzytelnieniu. Zawiera: AccessToken (token JWT), UserId, Username oraz Role. Szczególnie istotne jest to, czego ten obiekt nie zawiera: żadnych danych wrażliwych takich jak PasswordHash, Email czy RegisteredAt. Izolacja ta jest przykładem świadomego projektowania kontraktu API — klient otrzymuje wyłącznie informacje niezbędne do obsługi sesji.

3.3. DTOs warstwy filmów

CreateMovieRequest / UpdateMovieRequest

Obiekty żądań tworzenia i aktualizacji filmu. Oba zawierają identyczny zestaw pól: Title, Year, Description, PosterUrl (opcjonalne), GenreId oraz DirectorId. Przyjęcie identyfikatorów (GenreId, DirectorId) zamiast zagnieżdżonych obiektów jest celowym uproszczeniem kontraktu wejściowego — klient wskazuje istniejące encje przez ich klucze, a warstwa serwisowa odpowiada za weryfikację ich istnienia. Pola Id, Ratings ani żadne właściwości nawigacyjne nie są eksponowane w żądaniach, co eliminuje ryzyko mass assignment attacks.

MovieSummaryResponse

Obiekt odpowiedzi stosowany na listach filmów. Zawiera: Id, Title, Year, PosterUrl, zagnieżdżony obiekt MovieGenreSummaryResponse (z polami Id i Name) oraz DirectorSummaryResponse (z polami Id, FirstName, LastName, Nationality), a także obliczoną wartość AverageRating. Celowe pominięcie pola Description oraz kolekcji Ratings redukuje rozmiar odpowiedzi przy listowaniu wielu filmów, realizując zasadę minimalnego transferu danych.

MovieDetailResponse

Obiekt odpowiedzi dla widoku szczegółowego pojedynczego filmu. Rozszerza zestaw danych o pełny opis (Description), kompletną listę ocen (IReadOnlyList<RatingResponse>) oraz obliczoną średnią ocen (AverageScore). Zastosowanie IReadOnlyList<T> zamiast ICollection<T> lub List<T> wyraża semantyczną niezmiennosc kolekcji w kontekście odpowiedzi HTTP — klient nie powinien modyfikować tej kolekcji.

MovieListQueryRequest

Obiekt parametrów zapytania filtrującego listę filmów. Zawiera trzy opcjonalne pola: Title (filtrowanie po tytule), GenreId oraz DirectorId. Enkapsulacja parametrów filtrowania w dedykowanym obiekcie DTO zamiast przekazywania ich jako osobnych parametrów metody kontrolera zwiększa czytelność kodu i ułatwia rozszerzanie kryteriów filtrowania w przyszłości.

3.4. DTOs warstwy ocen

CreateRatingRequest / UpdateRatingRequest

Obiekty żądań tworzenia i modyfikacji oceny. Oba zawierają wyłącznie Value (wartość liczbowa) i opcjonalny Comment. Identyfikatory MovieId i UserId są celowo pominięte — MovieId jest pobierany z parametru ścieżki URL (route parameter), natomiast UserId jest wyodrębniony z tokenu JWT przez warstwę autoryzacji. Takie podejście eliminuje możliwość podszywania się pod innego użytkownika poprzez manipulację ciałem żądania.

RatingResponse

Obiekt odpowiedzi reprezentujący ocenę. Zawiera: Id, Value, Comment, CreatedAt, UserId oraz Username. Włączenie pola Username (zamiast wyłącznie UserId) jest przykładem denormalizacji na poziomie DTO — klient otrzymuje dane gotowe do wyświetlenia bez konieczności wykonywania dodatkowych zapytań.

3.5. DTOs warstwy reżyserów i gatunków

CreateDirectorRequest (DTOs/Directors/) — przyjmuje FirstName, LastName i Nationality. DirectorResponse — zwraca te same pola wraz z Id. Analogicznie, CreateGenreRequest przyjmuje wyłącznie Name, a GenreResponse zwraca Id i Name. Prostota tych obiektów odzwierciedla prostotę odpowiadających im encji domenowych.

3.6. Obiekty podsumowujące (Summary DTOs)

DirectorSummaryResponse i MovieGenreSummaryResponse są zagnieżdżanymi obiektami stosowanymi wewnątrz odpowiedzi filmowych. Ich wyodrębnienie jako osobnych klas zamiast bezpośredniego zagnieżdżania pól realizuje zasadę DRY (Don't Repeat Yourself) — te same struktury są wielokrotnie wykorzystywane w MovieSummaryResponse i MovieDetailResponse.

4. Architektura obsługi błędów — wyjątki domenowe

4.1. Filozofia projektowania

Projekt MovieHub implementuje architekturę obsługi błędów opartą na hierarchii własnych wyjątków domenowych, które semantycznie opisują rodzaj naruszenia reguł biznesowych lub stanu systemu. Podejście to stanowi alternatywę dla zwracania kodów błędów lub obiektów wynikowych (Result pattern) i jest powszechnie stosowane w aplikacjach ASP.NET Core, gdzie globalny middleware (np. `ExceptionHandler` lub `UseExceptionHandler`) przechwytyje wyjątki i mapuje je na odpowiednie odpowiedzi HTTP.

Wszystkie własne wyjątki dziedziczą bezpośrednio po klasie `System.Exception` i implementują trzy standardowe konstruktory: bezparametrowy, przyjmujący komunikat (string message) oraz przyjmujący komunikat i wyjątek wewnętrzny (string message, `Exception innerException`). Taka implementacja jest zgodna z wytycznymi projektowania wyjątków w platformie .NET i zapewnia pełną kompatybilność z mechanizmami serializacji i łańcuchowania wyjątków.

4.2. Katalog wyjątków i ich mapowanie na kody HTTP

NotFoundException

Wyjątek sygnalizujący, że żądany zasób nie istnieje w systemie. Typowe scenariusze użycia obejmują próbę pobrania, aktualizacji lub usunięcia encji o nieistniejącym identyfikatorze (np. film o podanym `Id` nie istnieje w bazie danych). Wyjątek ten mapuje się na kod statusu HTTP 404 Not Found, który zgodnie ze specyfikacją RFC 9110 oznacza, że serwer nie odnalazł zasobu odpowiadającego podanemu URI.

ConflictException

Wyjątek sygnalizujący konflikt stanu — próbę wykonania operacji, która naruszałaby integralność danych lub reguły biznesowe w kontekście aktualnego stanu systemu. Przykładem może być próba dodania oceny do filmu przez użytkownika, który już wcześniej wystawił ocenę temu filmowi (naruszenie unikalnego indeksu (`MoviId`, `UserId`)). Wyjątek ten mapuje się na kod statusu HTTP 409 Conflict, który informuje klienta, że żądanie nie może zostać zrealizowane z powodu konfliktu z aktualnym stanem zasobu.

DuplicateUserException

Wyjątek będący semantycznym uszczegółowieniem konfliktu w kontekście rejestracji użytkownika. Sygnalizuje próbę utworzenia konta z nazwą użytkownika (`Username`) lub adresem e-mail (`Email`), które już istnieją w systemie — co naruszałoby unikalne indeksy bazy danych. Choć semantycznie zbliżony do `ConflictException`, jego wyodrębnienie jako osobnej klasy umożliwia precyzyjniejsze komunikaty błędów oraz potencjalnie odmienne traktowanie przez middleware. Wyjątek ten mapuje się na kod statusu HTTP 409 Conflict.

InvalidCredentialsException

Wyjątek sygnalizujący niepowodzenie uwierzytelnienia z powodu podania błędnych danych logowania (nieprawidłowy adres e-mail lub hasło). Wyjątek ten mapuje się na kod statusu HTTP 401 Unauthorized. Istotne jest, że komunikat błędu zwracany klientowi powinien być celowo ogólny (np. "Nieprawidłowe dane logowania"), aby nie ujawniać, czy

błędny był adres e-mail, czy hasło — co stanowi zabezpieczenie przed atakami enumeracji użytkowników.

ForbiddenException

Wyjątek sygnalizujący, że uwierzytelniony użytkownik nie posiada uprawnień do wykonania żądanej operacji. Różnica pomiędzy `InvalidCredentialsException` (401) a `ForbiddenException` (403) jest fundamentalna z perspektywy semantyki HTTP: kod 401 oznacza brak lub nieprawidłowość danych uwierzytelniających, natomiast kod 403 oznacza, że tożsamość użytkownika jest znana i potwierdzona, lecz jego rola lub uprawnienia nie pozwalają na dostęp do zasobu. Przykładem użycia jest próba wykonania operacji administracyjnej przez użytkownika z rolą User zamiast Admin. Wyjątek ten mapuje się na kod statusu HTTP 403 Forbidden.

4.3. Tabela mapowania wyjątków na kody HTTP

Klasa wyjątku	Kod HTTP	Znaczenie
<code>NotFoundException</code>	404 Not Found	Zasób o podanym identyfikatorze nie istnieje
<code>ConflictException</code>	409 Conflict	Operacja narusza integralność lub stan systemu
<code>DuplicateUserException</code>	409 Conflict	Próba rejestracji z istniejącą nazwą lub e-mailem
<code>InvalidCredentialsException</code>	401 Unauthorized	Błędne dane logowania
<code>ForbiddenException</code>	403 Forbidden	Brak uprawnień do wykonania operacji

4.4. Integracja z pipeline'em ASP.NET Core

Opisane wyjątki domenowe nabierają pełnej wartości architektonicznej dopiero w połączeniu z globalnym mechanizmem obsługi wyjątków w pipeline'ie ASP.NET Core. Middleware przechwytyjący wyjątki (zaimplementowany jako `ExceptionHandler` lub przy użyciu metody `UseExceptionHandler`) odpowiada za translację wyjątków na ustrukturyzowane odpowiedzi HTTP zgodne ze standardem RFC 9457 (Problem Details for HTTP APIs).

Dzięki temu warstwa serwisowa może sygnalizować błędy w sposób semantycznie bogaty i niezależny od protokołu HTTP, podczas gdy odpowiedzialność za formatowanie odpowiedzi HTTP spoczywa wyłącznie na warstwie infrastrukturalnej. Takie podejście realizuje zasadę separacji odpowiedzialności (SoC) i znacząco upraszcza kod warstwy serwisowej, eliminując konieczność bezpośredniego operowania na obiektach `ActionResult` czy kodach statusu HTTP w logice biznesowej.

5. Serwisy — warstwa logiki biznesowej

Serwisy są jedynym miejscem, w którym wykonywana jest logika biznesowa projektu MovieHub. Każdy serwis jest wstrzykiwany do odpowiedniego kontrolera przez DI

(Dependency Injection) i operuje bezpośrednio na `AppDbContext` — bez warstwy repozytorium (wzorzec No Repository Pattern). Przepływ wywołań: `Controller` → `Service` → `DbContext` → `PostgreSQL`.

5.1. AuthService

`AuthService` jest odpowiedzialny za rejestrację nowych użytkowników oraz uwierzytelnianie istniejących. Wstrzykiwany jest do niego `AppDbContext` (dostęp do bazy danych) oraz `IOptions<JwtSettings>` (konfiguracja tokenów JWT z pliku `appsettings.json`).

RegisterAsync(RegisterRequest request, CancellationToken cancellationToken)

Metoda przyjmuje obiekt `RegisterRequest` zawierający pola `Username`, `Email` oraz `Password`. W pierwszym kroku wykonuje zapytanie do tabeli `Users` w bazie danych, sprawdzając za pomocą `AnyAsync`, czy istnieje już użytkownik o podanej nazwie użytkownika. Jeśli tak, rzuca wyjątek `DuplicateUserException` z komunikatem "Username is already taken.". Następnie analogicznie sprawdza unikalność adresu e-mail — w przypadku kolizji rzuca ten sam typ wyjątku z komunikatem "Email is already taken.".

Gdy oba pola są unikalne, tworzy nowy obiekt encji `User`, przypisując mu podane dane, haszując hasło przy użyciu biblioteki `BCrypt` (z domyślnym współczynnikiem kosztu 12), ustawiając rolę `UserRole.User` oraz znacznik czasu rejestracji `DateTime.UtcNow`. Nowy użytkownik jest dodawany do kontekstu i zapisywany w bazie przez `SaveChangesAsync`. Na końcu metoda wywołuje prywatną metodę `CreateAuthResponse`, która generuje token JWT i zwraca obiekt `AuthResponse`.

LoginAsync(LoginRequest request, CancellationToken cancellationToken)

Metoda przyjmuje obiekt `LoginRequest` zawierający pola `Email` oraz `Password`. Najpierw próbuje odnaleźć użytkownika w bazie danych po adresie e-mail za pomocą `FirstOrDefaultAsync`. Aby zapobiec atakom czasowym (timing attacks), niezależnie od tego, czy użytkownik istnieje, zawsze wykonywana jest operacja weryfikacji hasła `BCrypt` — jeśli użytkownik nie istnieje, weryfikacja odbywa się względem stałego, wbudowanego w kod fikcyjnego hasha (`dummyHash`), co sprawia, że czas odpowiedzi jest zbliżony w obu przypadkach. Jeśli użytkownik nie istnieje lub hasło jest nieprawidłowe, rzucony jest wyjątek `InvalidCredentialsException`. W przypadku pomyślnej weryfikacji metoda wywołuje `CreateAuthResponse` i zwraca token JWT wraz z danymi użytkownika.

GenerateJwtToken(User user) — metoda prywatna

Tworzy token JWT zawierający trzy claimy: `ClaimTypes.NameIdentifier` (identyfikator użytkownika), `ClaimTypes.Name` (nazwa użytkownika) oraz `ClaimTypes.Role` (rola użytkownika). Token jest podpisywany algorytmem `HMAC-SHA256` przy użyciu klucza symetrycznego odczytanego z konfiguracji. Czas wygaśnięcia tokenu jest obliczany jako `DateTime.UtcNow` powiększony o wartość `ExpirationMinutes` z konfiguracji. `Issuer` i `Audience` są ustawiane tylko wtedy, gdy nie są puste w konfiguracji.

5.2. MovieService

`MovieService` zarządza pełnym cyklem życia encji filmów — od pobierania list i szczegółów, przez tworzenie i aktualizację, aż po usuwanie. Wstrzykiwany jest wyłącznie `AppDbContext`.

GetAllAsync(MovieListQueryRequest query, CancellationToken cancellationToken)

Metoda przyjmuje obiekt zapytania `MovieListQueryRequest`, który może zawierać opcjonalne filtry: `Title` (string), `GenreId` (int?) oraz `DirectorId` (int?). Buduje zapytanie LINQ do tabeli `Movies` z flagą `AsNoTracking()` (brak śledzenia zmian, optymalizacja dla odczytu). Jeśli podano tytuł, filtruje rekordy przy użyciu `EF.Functions.ILike`, co zapewnia wyszukiwanie bez uwzględniania wielkości liter (case-insensitive), z dopasowaniem częściowym (`%title%`). Jeśli podano `GenreId` lub `DirectorId`, dodaje odpowiednie warunki `Where`. Wyniki są sortowane alfabetycznie po tytule.

Projekcja `Select` mapuje każdy film na obiekt `MovieSummaryResponse`, który zawiera podstawowe dane filmu, zagnieżdżony obiekt `MovieGenreSummaryResponse` z gatunkiem, zagnieżdżony obiekt `DirectorSummaryResponse` z reżyserem oraz wyliczoną średnią ocen (`AverageRating`) — jeśli film nie ma żadnych ocen, pole to przyjmuje wartość `null`. Całe zapytanie jest wykonywane jednorazowo po stronie bazy danych przez `ToListAsync`.

GetByIdAsync(int id, CancellationToken cancellationToken)

Pobiera pojedynczy film po jego identyfikatorze. Zapytanie używa `AsNoTracking()` i łączy powiązane encje przez `Include`: gatunek (`Genre`), reżysera (`Director`) oraz oceny (`Ratings`) wraz z użytkownikami, którzy je wystawili (`ThenInclude(r => r.User)`). Jeśli film o podanym id nie istnieje, rzuca wyjątek `NotFoundException`. W przypadku znalezienia rekordu, oceny są sortowane malejąco po dacie utworzenia i mapowane na listę obiektów `RatingResponse`. Obliczana jest średnia ocen (lub `null` jeśli brak ocen). Metoda zwraca bogaty obiekt `MovieDetailResponse` zawierający wszystkie dane filmu wraz z pełną listą ocen i komentarzy.

CreateAsync(CreateMovieRequest request, CancellationToken cancellationToken)

Przyjmuje obiekt `CreateMovieRequest` z polami: `Title`, `Year`, `Description`, `PosterUrl`, `GenreId` oraz `DirectorId`. Tworzy nową encję `Movie`, dodaje ją do kontekstu i zapisuje w bazie. Po zapisaniu wykonuje dodatkowe zapytanie do bazy danych, aby pobrać nowo utworzony film wraz z powiązanymi encjami gatunku i reżysera (niezbędne, bo EF Core nie wypełnia nawigacyjnych właściwości automatycznie po `Add`). Zwraca obiekt `MovieSummaryResponse`. Pole `AverageRating` w odpowiedzi jest zawsze `null`, ponieważ nowo dodany film nie ma jeszcze żadnych ocen.

UpdateAsync(int id, UpdateMovieRequest request, CancellationToken cancellationToken)

Pobiera film z bazy danych po id — tym razem bez `AsNoTracking()`, ponieważ encja musi być śledzona przez EF Core w celu zapisania zmian. Jeśli film nie istnieje, rzuca `NotFoundException`. Następnie nadpisuje wszystkie pola encji wartościami z obiektu `UpdateMovieRequest` (pełna aktualizacja, nie częściowa). Po wywołaniu `SaveChangesAsync` wykonuje kolejne zapytanie z `Include` dla gatunku i reżysera, aby zwrócić kompletny obiekt `MovieSummaryResponse`.

DeleteAsync(int id, CancellationToken cancellationToken)

Pobiera film po id. Jeśli nie istnieje, rzuca `NotFoundException`. W przeciwnym razie wywołuje `_dbContext.Movies.Remove(movie)` i zapisuje zmiany. Metoda nie zwraca żadnej wartości (`Task`).

5.3. DirectorService

DirectorService obsługuje operacje na encjach reżyserów. Jest to prosty serwis z dwoma metodami publicznymi, bez rozbudowanej logiki walidacji.

GetAllAsync pobiera wszystkich reżyserów z tabeli Directors z flagą AsNoTracking(). Wyniki są sortowane rosnąco po nazwisku (LastName). Każdy rekord jest mapowany bezpośrednio w zapytaniu LINQ na obiekt DirectorResponse zawierający pola Id, FirstName, LastName oraz Nationality. Metoda zwraca Task<List<DirectorResponse>> bez dodatkowego await — zapytanie jest zwracane jako gotowy task z ToListAsync.

CreateAsync przyjmuje obiekt CreateDirectorRequest z polami FirstName, LastName oraz Nationality. Tworzy nową encję Director, dodaje ją do kontekstu i zapisuje w bazie. Nie sprawdza unikalności danych — możliwe jest dodanie dwóch reżyserów o identycznych danych. Po zapisaniu mapuje encję na DirectorResponse przy użyciu prywatnej metody MapToResponse i zwraca wynik.

5.4. GenreService

GenreService zarządza gatunkami filmowymi. W odróżnieniu od DirectorService posiada walidację unikalności nazwy gatunku.

GetAllAsync pobiera wszystkie gatunki z tabeli Genres z flagą AsNoTracking(). Wyniki są sortowane rosnąco alfabetycznie po nazwie (Name). Każdy rekord jest mapowany na obiekt GenreResponse zawierający pola Id oraz Name. Podobnie jak w DirectorService, metoda zwraca task bezpośrednio z ToListAsync.

CreateAsync przyjmuje obiekt CreateGenreRequest z polem Name. Przed utworzeniem gatunku sprawdza, czy w bazie danych istnieje już gatunek o tej samej nazwie (porównanie dokładne, bez ILike — uwzględnia wielkość liter). Jeśli nazwa jest zajęta, rzuca wyjątek ConflictException z komunikatem "Genre name is already taken.". W przypadku unikalnej nazwy tworzy nową encję Genre, zapisuje ją w bazie i zwraca zmapowany obiekt GenreResponse.

5.5. RatingService

RatingService obsługuje tworzenie i aktualizację ocen filmów przez zalogowanych użytkowników. Zawiera logikę zapobiegającą wielokrotnemu ocenianiu tego samego filmu przez jednego użytkownika.

CreateAsync(int movieId, int userId, CreateRatingRequest request, CancellationToken cancellationToken)

Przyjmuje identyfikator filmu, identyfikator użytkownika (wyodrębniony z tokenu JWT przez kontroler) oraz obiekt CreateRatingRequest z polami Value (ocena numeryczna) i Comment (opcjonalny komentarz). W pierwszym kroku pobiera film z bazy danych wraz z jego ocenami (Include(m => m.Ratings)). Jeśli film nie istnieje, rzuca NotFoundException. Następnie pobiera użytkownika po userId — jeśli nie istnieje, rzuca NotFoundException. Sprawdza, czy użytkownik nie wystawił już oceny dla tego filmu (movie.Ratings.Any(r => r.UserId == userId)); jeśli tak, rzuca ConflictException z komunikatem "User already rated this movie.". Jeśli wszystkie warunki są spełnione, tworzy nową encję Rating z aktualnym znacznikiem czasu UTC, zapisuje ją w bazie i zwraca obiekt RatingResponse z nazwą użytkownika przekazaną bezpośrednio (bez dodatkowego zapytania do bazy).

UpdateAsync(int userId, int ratingId, UpdateRatingRequest request, CancellationToken cancellationToken)

Przyjmuje identyfikator użytkownika (z tokenu JWT), identyfikator oceny do zaktualizowania oraz obiekt UpdateRatingRequest z polami Value i Comment. Pobiera

ocenę z bazy danych wraz z powiązaniem użytkownikiem (`Include(r => r.User)`). Jeśli ocena nie istnieje, rzuca `NotFoundException`. Następnie weryfikuje, czy `rating.UserId` odpowiada `userId` z tokenu — jeśli nie, rzuca `ForbiddenException` z komunikatem "You are not allowed to update this rating.", co zapobiega modyfikowaniu cudzych ocen. Po pozytywnej weryfikacji nadpisuje pola `Value` i `Comment`, zapisuje zmiany i zwraca zaktualizowany obiekt `RatingResponse`.

6. Kontrolery — warstwa interfejsu API

Kontrolery odpowiadają wyłącznie za obsługę HTTP: parsowanie żądań, wywoływanie serwisów i zwracanie odpowiednich kodów statusu. Cała logika biznesowa jest delegowana do serwisów. Identyfikator użytkownika jest wyodrębniony w kontrolerze przez `User.FindFirst(ClaimTypes.NameIdentifier)`. Endpointy administracyjne wymagają atrybutu `[Authorize(Roles = "Admin")]`.

6.1. AuthController — trasa bazowa: /api/auth

Kontroler obsługuje rejestrację i logowanie użytkowników. Nie wymaga autoryzacji dla żadnego z endpointów.

POST /api/auth/register

Przeznaczenie: Rejestracja nowego konta użytkownika w systemie.

Oczekiwane DTO (ciało żądania): `RegisterRequest`

- `Username` (string) — unikalna nazwa użytkownika
- `Email` (string) — unikalny adres e-mail
- `Password` (string) — hasło w postaci jawnej (zostanie zahaszkowane)

Kod statusu	Znaczenie
201 Created	Rejestracja zakończona sukcesem; w ciele odpowiedzi zwracany jest obiekt <code>AuthResponse</code> zawierający <code>AccessToken</code> (token JWT), <code>UserId</code> , <code>Username</code> oraz <code>Role</code>
409 Conflict	Nazwa użytkownika lub adres e-mail są już zajęte; w ciele odpowiedzi obiekt z polem <code>message</code> opisującym przyczynę konfliktu

POST /api/auth/login

Przeznaczenie: Uwierzytelnienie istniejącego użytkownika i uzyskanie tokenu JWT.

Oczekiwane DTO (ciało żądania): `LoginRequest`

- `Email` (string) — adres e-mail użytkownika
- `Password` (string) — hasło w postaci jawnej

Kod statusu	Znaczenie
200 OK	Logowanie zakończone sukcesem; w ciele odpowiedzi zwracany jest obiekt AuthResponse z tokenem JWT i danymi użytkownika
401 Unauthorized	Nieprawidłowy adres e-mail lub hasło; w ciele odpowiedzi obiekt z polem message

6.2. MoviesController — trasa bazowa: /api/movies

Kontroler obsługuje pełny CRUD dla filmów. Operacje odczytu są dostępne publicznie, natomiast tworzenie, aktualizacja i usuwanie wymagają roli Admin.

GET /api/movies

Przeznaczenie: Pobranie listy wszystkich filmów z opcjonalnym filtrowaniem.

Oczekiwane parametry zapytania ([FromQuery]): MovieListQueryRequest

- Title (string, opcjonalny) — filtrowanie po tytule (wyszukiwanie częściowe, bez uwzględniania wielkości liter)
- GenreId (int?, opcjonalny) — filtrowanie po identyfikatorze gatunku
- DirectorId (int?, opcjonalny) — filtrowanie po identyfikatorze reżysera

Kod statusu	Znaczenie
200 OK	Lista obiektów MovieSummaryResponse (może być pusta); każdy obiekt zawiera Id, Title, Year, PosterUrl, zagnieżdżony gatunek, zagnieżdżonego reżysera oraz AverageRating

GET /api/movies/{id}

Przeznaczenie: Pobranie szczegółowych danych pojedynczego filmu wraz z pełną listą ocen i komentarzy.

Parametry ścieżki: id (int) — identyfikator filmu.

Kod statusu	Znaczenie
200 OK	Obiekt MovieDetailResponse zawierający wszystkie pola filmu, zagnieżdżony gatunek, reżysera, listę obiektów RatingResponse (posortowanych malejąco po dacie) oraz AverageScore
404 Not Found	Film o podanym id nie istnieje; w ciele odpowiedzi obiekt z polem message

POST /api/movies — [Authorize(Roles = "Admin")]

Przeznaczenie: Dodanie nowego filmu do bazy danych. Wymaga roli Admin.

Oczekiwane DTO (ciało żądania): CreateMovieRequest — Title, Year, Description, PosterUrl, GenreId, DirectorId.

Kod statusu	Znaczenie
-------------	-----------

201 Created	Film został utworzony; w ciele odpowiedzi obiekt MovieSummaryResponse
401 Unauthorized	Brak tokenu JWT lub token jest nieważny
403 Forbidden	Użytkownik jest zalogowany, ale nie posiada roli Admin

PUT /api/movies/{id} — [Authorize(Roles = "Admin")]

Przeznaczenie: Pełna aktualizacja danych istniejącego filmu. Wymaga roli Admin.

Parametry ścieżki: id (int) — identyfikator filmu do zaktualizowania. Oczekiwane DTO: UpdateMovieRequest — Title, Year, Description, PosterUrl, GenreId, DirectorId.

Kod statusu	Znaczenie
200 OK	Film został zaktualizowany; w ciele odpowiedzi obiekt MovieSummaryResponse z nowymi danymi
404 Not Found	Film o podanym id nie istnieje
401 Unauthorized	Brak lub nieważny token JWT
403 Forbidden	Użytkownik nie posiada roli Admin

DELETE /api/movies/{id} — [Authorize(Roles = "Admin")]

Przeznaczenie: Usunięcie filmu z bazy danych. Wymaga roli Admin.

Parametry ścieżki: id (int) — identyfikator filmu do usunięcia.

Kod statusu	Znaczenie
204 No Content	Film został pomyślnie usunięty; brak ciała odpowiedzi
404 Not Found	Film o podanym id nie istnieje
401 Unauthorized	Brak lub nieważny token JWT
403 Forbidden	Użytkownik nie posiada roli Admin

6.3. DirectorsController — trasa bazowa: /api/directors

Kontroler obsługuje pobieranie listy reżyserów oraz dodawanie nowych. Brak endpointów do aktualizacji i usuwania.

GET /api/directors

Przeznaczenie: Pobranie listy wszystkich reżyserów posortowanych alfabetycznie po nazwisku. Endpoint publiczny, nie wymaga autoryzacji.

Kod statusu	Znaczenie
200 OK	Lista obiektów DirectorResponse, każdy zawierający Id, FirstName, LastName oraz Nationality; lista może być pusta

POST /api/directors — [Authorize(Roles = "Admin")]

Przeznaczenie: Dodanie nowego reżysera do bazy danych. Wymaga roli Admin.

Oczekiwane DTO (ciało żądania): CreateDirectorRequest — FirstName, LastName, Nationality.

Kod statusu	Znaczenie
201 Created	Reżyser został dodany; w ciele odpowiedzi obiekt DirectorResponse
401 Unauthorized	Brak lub nieważny token JWT
403 Forbidden	Użytkownik nie posiada roli Admin

6.4. GenresController — trasa bazowa: /api/genres

Kontroler obsługuje pobieranie listy gatunków oraz dodawanie nowych. Brak endpointów do aktualizacji i usuwania.

GET /api/genres

Przeznaczenie: Pobranie listy wszystkich gatunków filmowych posortowanych alfabetycznie po nazwie. Endpoint publiczny, nie wymaga autoryzacji.

Kod statusu	Znaczenie
200 OK	Lista obiektów GenreResponse, każdy zawierający Id oraz Name; lista może być pusta

POST /api/genres — [Authorize(Roles = "Admin")]

Przeznaczenie: Dodanie nowego gatunku filmowego. Wymaga roli Admin. Nazwa gatunku musi być unikalna.

Oczekiwane DTO (ciało żądania): CreateGenreRequest — Name (musi być unikalna w bazie danych).

Kod statusu	Znaczenie
201 Created	Gatunek został dodany; w ciele odpowiedzi obiekt GenreResponse

409 Conflict	Gatunek o podanej nazwie już istnieje; w ciele odpowiedzi obiekt z polem message
401 Unauthorized	Brak lub nieważny token JWT
403 Forbidden	Użytkownik nie posiada roli Admin

6.5. RatingsController — trasa bazowa: /api

Kontroler obsługuje wystawianie i aktualizację ocen filmów. Oba endpointy wymagają zalogowania (dowolna rola). Identyfikator użytkownika jest każdorazowo odczytywany z claimu ClaimTypes.NameIdentifier w tokenie JWT — użytkownik nie może podać cudzego userId.

POST /api/movies/{movieId}/ratings — [Authorize]

Przeznaczenie: Wystawienie oceny dla wskazanego filmu przez zalogowanego użytkownika. Każdy użytkownik może wystawić tylko jedną ocenę dla danego filmu.

Parametry ścieżki: movieId (int) — identyfikator ocenianego filmu.

Oczekiwane DTO (ciało żądania): CreateRatingRequest — Value (numeryczny), Comment (string, opcjonalny).

Kod statusu	Znaczenie
201 Created	Ocena została dodana; w ciele odpowiedzi obiekt RatingResponse zawierający Id, Value, Comment, CreatedAt, UserId oraz Username
401 Unauthorized	Brak lub nieważny token JWT
404 Not Found	Film lub użytkownik o podanym identyfikatorze nie istnieje (obsługiwane przez globalny middleware wyjątków)
409 Conflict	Użytkownik już wystawił ocenę dla tego filmu (obsługiwane przez globalny middleware wyjątków)

PUT /api/ratings/{ratingId} — [Authorize]

Przeznaczenie: Aktualizacja istniejącej oceny przez jej autora. Użytkownik może zaktualizować wyłącznie własną ocenę — próba modyfikacji cudzej oceny skutkuje błędem 403 Forbidden.

Parametry ścieżki: ratingId (int) — identyfikator oceny do zaktualizowania.

Oczekiwane DTO (ciało żądania): UpdateRatingRequest — Value (numeryczny), Comment (string, opcjonalny).

Kod statusu	Znaczenie
-------------	-----------

200 OK	Ocena została zaktualizowana; w ciele odpowiedzi obiekt RatingResponse z nowymi danymi
401 Unauthorized	Brak tokenu JWT lub brak claimu z userId (kontroler zwraca Unauthorized bezpośrednio, bez rzucania wyjątku)
403 Forbidden	Zalogowany użytkownik nie jest autorem wskazanej oceny (obsługiwane przez globalny middleware wyjątków na podstawie ForbiddenException)
404 Not Found	Ocena o podanym ratingId nie istnieje (obsługiwane przez globalny middleware wyjątków)

6.6. Pełna tabela endpointów API

Metoda	Endpoint	Autoryzacja	Sukces
POST	/api/auth/register	Publiczny	201 Created
POST	/api/auth/login	Publiczny	200 OK
GET	/api/movies	Publiczny	200 OK
GET	/api/movies/{id}	Publiczny	200 OK
POST	/api/movies	Admin	201 Created
PUT	/api/movies/{id}	Admin	200 OK
DELETE	/api/movies/{id}	Admin	204 No Content
POST	/api/movies/{id}/ratings	Użytkownik	201 Created
PUT	/api/ratings/{id}	Użytkownik	200 OK
GET	/api/genres	Publiczny	200 OK
POST	/api/genres	Admin	201 Created
GET	/api/directors	Publiczny	200 OK
POST	/api/directors	Admin	201 Created

7. Architektura frontendu — Avalonia MVVM

Podprojekt kliencki aplikacji MovieHub został zrealizowany w oparciu o framework Avalonia UI — wieloplatformową bibliotekę do budowania interfejsów graficznych w ekosystemie .NET, stanowiącą odpowiednik technologii WPF dla środowisk innych niż Windows. Architektura klienta opiera się na wzorcu projektowym MVVM (Model–View–ViewModel), który zakłada ścisłe rozdzielenie warstwy prezentacji (widoki, pliki .axaml) od logiki prezentacji (modele widoków, klasy *ViewModel) oraz od warstwy danych i usług (serwisy, modele domenowe).

Infrastruktura aplikacji klienta opiera się na czterech filarach:

- Kontener wstrzykiwania zależności (Microsoft.Extensions.DependencyInjection) zarządzający cyklem życia wszystkich usług i modeli widoków
- ViewLocator realizujący automatyczne wiązanie modeli widoków z odpowiadającymi im widokami na podstawie konwencji nazewnictwa
- NavigationService odpowiedzialny za dynamiczną podmianę aktywnego widoku w obrębie jednego okna aplikacji
- SessionService pełniący rolę centralnego magazynu stanu sesji użytkownika, przechowującego token uwierzytelniający oraz dane tożsamościowe

7.1. Cykl życia i proces uruchamiania aplikacji

Klasa Program — punkt wejścia i konfiguracja platformy

Klasa Program stanowi absolutny punkt wejścia procesu aplikacji. Jej metoda statyczna Main jest oznaczona atrybutem [STAThread], co jest wymogiem technicznym dla aplikacji okienkowych działających w modelu jednowątkowego apartamentu COM (Single-Threaded Apartment). Bez tego atrybutu mechanizmy obsługi schowka systemowego, przeciągania i upuszczania obiektów oraz inne operacje wymagające dostępu do zasobów COM mogłyby działać nieprawidłowo lub powodować wyjątki w czasie wykonania.

Metoda Main deleguje całość inicjalizacji do łańcucha wywołań zbudowanego przez metodę BuildAvaloniaApp(), a następnie uruchamia pętlę zdarzeń za pomocą StartWithClassicDesktopLifetime(args). Metoda BuildAvaloniaApp() jest celowo wyodrębniona jako publiczna metoda statyczna — jest ona bowiem wywoływana nie tylko w czasie rzeczywistego uruchomienia aplikacji, lecz również przez projektanta wizualnego (visual designer) zintegrowanego ze środowiskiem programistycznym, który potrzebuje dostępu do skonfigurowanego budowniczego aplikacji bez uruchamiania pełnego cyklu życia.

Łańcuch konfiguracji platformy obejmuje następujące kroki:

- `AppBuilder.Configure<App>()` — wskazuje klasę App jako główną klasę aplikacji Avalonia
- `.UsePlatformDetect()` — automatyczne wykrycie systemu operacyjnego i załadowanie odpowiedniego backendu renderowania (Win32 na Windows, X11 lub Wayland na Linux, AppKit na macOS)
- `.WithDeveloperTools()` — włączane wyłącznie w konfiguracji DEBUG; udostępnia narzędzia deweloperskie Avalonia (inspektor drzewa wizualnego, podgląd właściwości), analogiczne do DevTools w przeglądarkach internetowych
- `.WithInterFont()` — rejestruje krój pisma Inter jako domyślną czcionkę aplikacji, zapewniając spójny wygląd typograficzny niezależnie od zainstalowanych czcionek systemowych
- `.LogToTrace()` — kieruje komunikaty diagnostyczne frameworka do standardowego mechanizmu `System.Diagnostics.Trace`

Klasa App — rejestracja zależności i inicjalizacja głównego okna

Klasa App, dziedzicząca po `Avalonia.Application`, pełni rolę centralnego orkiestratora inicjalizacji aplikacji. Metoda `Initialize()` jest wywoływana jako pierwsza i ładuje zasoby XAML zdefiniowane w pliku `App.axaml` za pomocą wywołania `AvaloniaXamlLoader.Load(this)`. Na tym etapie rejestrowane są globalne zasoby aplikacji,

takie jak słowniki stylów, pędzle, szablony danych oraz instancja klasy ViewLocator jako globalny szablon danych (DataTemplate).

Metoda OnFrameworkInitializationCompleted() realizuje trzy zasadnicze operacje:

- Budowa kontenera IoC — tworzona jest instancja ServiceCollection, do której rejestrowane są wszystkie zależności aplikacji za pomocą metody pomocniczej ConfigureServices. Następnie kolekcja jest kompilowana do niezmiennego dostawcy usług (IServiceProvider) i przechowywana w statycznej właściwości App.Services. Udostępnienie dostawcy jako właściwości statycznej jest świadomym kompromisem architektonicznym — Avalonia nie posiada wbudowanego mechanizmu wstrzykiwania zależności do konstruktorów widoków XAML.
- Rejestracja usług — HttpClient jako singleton z predefiniowanym adresem bazowym, ApiClient jako singleton, SessionService jako singleton, NavigationService jako singleton z fabryczną funkcją rozwiązywania typów (type => sp.GetRequiredService(type)), MainWindowViewModel jako singleton, *ViewModel jako transient — nowe instancje tworzone przy każdym żądaniu nawigacji.
- Tworzenie głównego okna — po zbudowaniu kontenera tworzona jest instancja MainWindow, której właściwość DataContext jest ustawiana na MainWindowViewModel pobrany z kontenera.

7.2. Mechanizm ViewLocator — automatyczne wiązanie widoków z modelami widoków

Rola interfejsu IDataTemplate

Klasa ViewLocator implementuje interfejs IDataTemplate, który jest fundamentalnym mechanizmem Avalonia pozwalającym na dynamiczne generowanie kontrolki na podstawie danych. Gdy silnik renderowania Avalonia napotyka obiekt przypisany do właściwości DataContext lub Content kontrolki i nie posiada dla niego jawnie zdefiniowanego szablonu danych, przeszukuje hierarchię zasobów w poszukiwaniu implementacji IDataTemplate, której metoda Match zwróci wartość true dla danego obiektu.

Konwencja nazewnicza jako mechanizm wiązania

Metoda Match klasy ViewLocator zawiera logikę kwalifikacyjną: zwraca true wyłącznie dla obiektów będących instancjami klasy ViewModelBase lub jej klas pochodnych. Metoda Build realizuje właściwy mechanizm rozwiązywania widoku — pobiera w pełni kwalifikowaną nazwę typu modelu widoku (np.

MovieHub.Client.ViewModels.BrowseViewModel), a następnie dokonuje leksykalnego podstawienia ciągu "ViewModel" na "View", uzyskując nazwę

MovieHub.Client.Views.BrowseView. Następnie za pomocą refleksji (Type.GetType) próbuje odnaleźć typ o tej nazwie w załadowanych zestawach. Jeśli typ zostanie znaleziony, tworzona jest jego instancja za pomocą Activator.CreateInstance. W przypadku niepowodzenia zwracany jest kontrolka TextBlock z komunikatem diagnostycznym "Not Found: <nazwa>", co znacząco ułatwia debugowanie podczas tworzenia nowych widoków.

Atrybut [RequiresUnreferencedCode] informuje narzędzia do przycinania kodu (tree shaking, np. używane przy publikacji AOT) o tym, że klasa korzysta z refleksji, której wyniki nie mogą być statycznie przewidziane w czasie kompilacji. Jest to ważna adnotacja dla scenariuszy publikacji z włączoną opcją PublishTrimmed.

Mechanizm ViewLocator eliminuje konieczność jawnego definiowania szablonów danych (DataTemplate) dla każdej pary ViewModel–View. Modele widoków pozostają całkowicie nieświadome istnienia konkretnych klas widoków. Dodanie nowej pary ViewModel–View do aplikacji wymaga jedynie przestrzegania konwencji nazewnictwa, bez modyfikacji jakiegokolwiek centralnego rejestru.

7.3. CommunityToolkit.Mvvm — mechanizm generowania kodu

Projekt MovieHub.Client w całości opiera warstwę prezentacji na bibliotece CommunityToolkit.Mvvm, która działa jako generator kodu źródłowego w czasie kompilacji (Roslyn Source Generator). Oznacza to, że programista deklaruje jedynie intencję za pomocą atrybutów, a kompilator automatycznie wstrzykuje do klasy pełną implementację wzorców reaktywnych — bez konieczności ręcznego pisania boilerplate'u.

[ObservableProperty]

Każde pole prywatne oznaczone atrybutem [ObservableProperty] powoduje wygenerowanie przez kompilator: publicznej właściwości o nazwie w PascalCase (np. pole `_searchTitle` → właściwość `SearchTitle`), implementacji `INotifyPropertyChanged` — wywołania `OnPropertyChanged` w seterze — oraz opcjonalnych metod częściowych `partial void OnSearchTitleChanging(string value)` i `partial void OnSearchTitleChanged(string value)`, które programista może nadpisać, aby reagować na zmianę wartości.

Dzięki temu silnik bindingów Avalonia automatycznie odświeża elementy UI powiązane z daną właściwością w momencie jej zmiany — bez żadnego dodatkowego kodu po stronie dewelopera.

[NotifyCanExecuteChangedFor]

Rozszerzenie [ObservableProperty] — gdy pole zmienia wartość, automatycznie wywołuje `CanExecuteChanged` na wskazanej komendzie. W `AuthViewModel` każde pole formularza jest oznaczone `[NotifyCanExecuteChangedFor(nameof(SubmitCommand))]`. Oznacza to, że za każdym razem, gdy użytkownik wpisze literę w dowolnym polu, Avalonia ponownie ewaluuje metodę `CanSubmit()` i odpowiednio włącza lub wyłącza przycisk `Submit` — bez żadnego ręcznego wywołania `RaiseCanExecuteChanged`.

[RelayCommand]

Metoda prywatna oznaczona [RelayCommand] powoduje wygenerowanie publicznej właściwości implementującej `ICommand` (lub `ISyncRelayCommand` dla metod asynchronicznych). Nazwa komendy to nazwa metody z sufiksem `Command` (np. metoda `SearchAsync()` → komenda `SearchCommand`). Wygenerowana komenda dla metod asynchronicznych implementuje `ISyncRelayCommand`, co oznacza automatyczną obsługę `IsRunning` i zapobieganie wielokrotnemu wywołaniu podczas trwania operacji.

Parametr `CanExecute = nameof(CanSubmit)` w `[RelayCommand(CanExecute = nameof(CanSubmit))]` wskazuje metodę strażnika — komenda jest aktywna tylko wtedy, gdy `CanSubmit()` zwraca `true`.

Klasa bazowa ViewModelBase

```
public abstract class ViewModelBase : ObservableObject { }
```

ObservableObject z CommunityToolkit.Mvvm to bazowa implementacja INotifyPropertyChanged i INotifyPropertyChanging. Wszystkie główne ViewModele dziedziczą po ViewModelBase, natomiast sub-ViewModele panelu Admin dziedziczą bezpośrednio po ObservableObject — co jest równoważne, ale pomija pośrednią warstwę abstrakcji projektu.

7.4. Serwisy infrastrukturalne

SessionService

SessionService pełni rolę centralnego, aplikacyjnego magazynu stanu sesji użytkownika. Zarejestrowany jako singleton w kontenerze IoC, istnieje przez cały czas życia procesu aplikacji i jest współdzielony przez wszystkie komponenty, które go potrzebują — w szczególności przez ApiClient, modele widoków oraz NavigationService.

Klasa przechowuje cztery właściwości stanu sesji (wszystkie z modyfikatorem dostępu private set — zewnętrzne komponenty mogą jedynie odczytywać stan):

- Token (string?) — ciąg znaków reprezentujący token JWT wydany przez serwer API po pomyślnym uwierzytelnieniu; jest następnie dołączany do nagłówka Authorization każdego żądania HTTP
- Username (string?) — nazwa wyświetlana zalogowanego użytkownika, wykorzystywana w interfejsie użytkownika do personalizacji komunikatów
- Role (string?) — rola przypisana użytkownikowi przez serwer (np. "Admin" lub rola standardowego użytkownika), determinująca zakres dostępnych funkcji
- UserId (int) — numeryczny identyfikator użytkownika w bazie danych, niezbędny przy operacjach wymagających identyfikacji zasobu (np. wystawianie ocen filmów)

Klasa udostępnia dwie właściwości obliczeniowe: IsLoggedIn => Token != null oraz IsAdmin => Role == "Admin". Metoda SetSession jest wywoływana przez AuthViewModel bezpośrednio po otrzymaniu odpowiedzi z serwera. Metoda Clear() realizuje wylogowanie poprzez wyzerowanie Token, Username i Role.

Uwaga: UserId nie jest jawnie zerowane w metodzie Clear — jest to potencjalny obszar do przyszłego doprecyzowania, choć w praktyce wartość UserId bez towarzyszącego tokenu nie stanowi zagrożenia bezpieczeństwa. SessionService przechowuje dane wyłącznie w pamięci operacyjnej procesu — zamknięcie aplikacji wymaga ponownego uwierzytelnienia. Jest to świadomy wybór projektowy, upraszczający implementację kosztem komfortu użytkownika.

NavigationService

NavigationService realizuje wzorzec nawigacji charakterystyczny dla aplikacji jednostronicowych (Single-Page Application), adaptowany do środowiska aplikacji desktopowych. Zamiast otwierać nowe okna, serwis zarządza pojedynczą właściwością reaktywną CurrentView, której zmiana powoduje automatyczne odświeżenie obszaru treści w głównym oknie aplikacji.

Klasa dziedziczy po ObservableObject, co zapewnia implementację INotifyPropertyChanged. Właściwość CurrentView jest powiązana z właściwością Content kontrolki ContentControl w widoku MainWindow. Konstruktor NavigationService przyjmuje parametr typu Func<Type, object>, będący delegatem fabrycznym — realizuje to wzorzec Service Locator w ograniczonym, kontrolowanym zakresie. NavigationService nie zna konkretnych typów modeli widoków w czasie kompilacji, lecz potrafi je tworzyć na żądanie, przekazując odpowiedzialność do kontenera IoC. Ograniczenie generyczne where

TViewModel : class zapewnia, że metoda może być wywołana wyłącznie dla typów referencyjnych, co jest zgodne z wymaganiami kontenera IoC.

Metoda `NavigateToMovie(int movieId)` po utworzeniu `MovieDetailViewModel` ustawia `CurrentView = viewModel`, a następnie przypisuje `detailVm.MovieId = movieId`. Warto zauważyć, że `CurrentView` jest ustawiany przed przypisaniem `MovieId`, co oznacza, że widok może zostać wyrenderowany chwilę przed załadowaniem danych — jest to typowy wzorec dla asynchronicznego ładowania danych, gdzie model widoku reaguje na zmianę `MovieId` inicjując żądanie do API.

ApiClient

`ApiClient` stanowi dedykowaną fasadę (facade) nad klasą `HttpClient`, enkapsulującą całość komunikacji sieciowej z serwerem REST API MovieHub. Zarejestrowany jako singleton, współdzieli jedną instancję `HttpClient` przez cały czas życia aplikacji, co jest zgodne z zaleceniami Microsoftu dotyczącymi zarządzania połączeniami HTTP i unikania wyczerpania puli gniazd sieciowych (socket exhaustion).

Klasa jest zorganizowana w logiczne grupy metod odpowiadające zasobom API: uwierzytelnianie (`LoginAsync`, `RegisterAsync`), filmy (`GetMoviesAsync`, `GetMovieAsync`, `CreateMovieAsync`, `UpdateMovieAsync`, `DeleteMovieAsync`), oceny (`CreateRatingAsync`, `UpdateRatingAsync`), gatunki (`GetGenresAsync`, `CreateGenreAsync`), reżyserzy (`GetDirectorsAsync`, `CreateDirectorAsync`).

Metoda `SetAuthToken(string token)` ustawia nagłówek `Authorization: Bearer <token>` na poziomie `DefaultRequestHeaders`, co oznacza, że wszystkie kolejne żądania HTTP automatycznie zawierają token autoryzacyjny. Wszystkie metody stosują wzorec `response.EnsureSuccessStatusCode()`. Deserializacja odpowiedzi JSON jest realizowana za pomocą `ReadFromJsonAsync<T>()` z przestrzeni nazw `System.Net.Http.Json`.

7.5. Lokalna obsługa błędów — PasswordMissMatchException

Folder `Exceptions` w projekcie klienckim zawiera definicje wyjątków specyficznych dla domeny aplikacji klienckiej. Zdefiniowany jest wyjątek `PasswordMissMatchException`, dziedziczący bezpośrednio po `System.Exception` i implementujący trzy konstruktory zgodnie z konwencją projektowania wyjątków w .NET (exception constructor pattern).

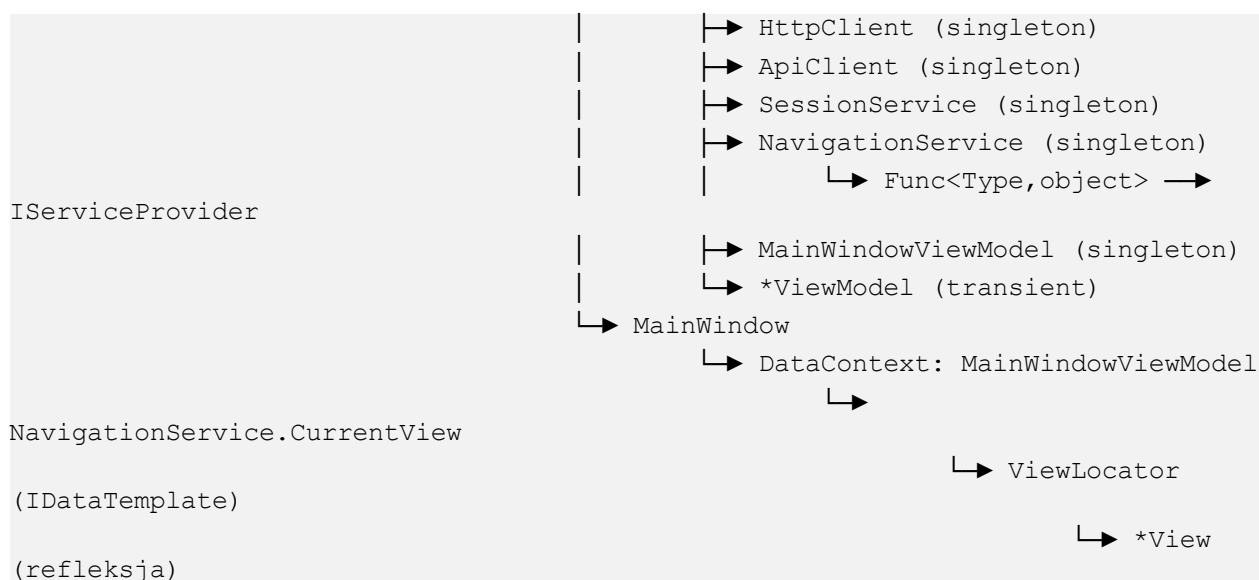
```
public class PasswordMissMatchException : Exception
{
    public PasswordMissMatchException(string message) : base(message) { }
    public PasswordMissMatchException() : base() { }
    public PasswordMissMatchException(string? message, Exception? innerException)
        : base(message, innerException) { }
}
```

Semantycznie wyjątek ten jest rzucany po stronie klienta — w modelu widoku formularza rejestracji — w momencie, gdy użytkownik wprowadzi dwa różne hasła w polach "hasło" i "powtórz hasło". Jest to przykład walidacji domenowej po stronie klienta, realizowanej przed wysłaniem żądania do serwera. Zastosowanie dedykowanego wyjątku zamiast prostego warunku `if` z komunikatem błędu świadczy o dążeniu do ekspresywności kodu i możliwości centralnej obsługi błędów walidacji w przyszłości.

7.6. Diagram zależności infrastruktury klienta

Program.cs

```
└─ BuildAvaloniaApp() ──> App.axaml.cs
                           └─> ConfigureServices()
```



8. Moduły prezentacji — ViewModele i widoki

8.1. Moduł Auth — logowanie i rejestracja

AuthViewModel

AuthViewModel jest wstrzykiwany z trzema zależnościami: ApiClient, SessionService i NavigationService. Zarządza jednym formularzem, który pełni dwie role — logowania i rejestracji — przełączanych przez zakładki.

Właściwości obserwowalne i ich semantyka:

- `IsLoginTab` (bool, domyślnie true) — flaga określająca aktywny tryb. Zmiana tej właściwości powoduje (przez `[NotifyCanExecuteChangedFor(nameof(SubmitCommand))]`) ponowną ewaluację `CanSubmit()`.
- `Email`, `Username`, `Password`, `PasswordRegister` (string) — pola formularza, wszystkie z `[NotifyCanExecuteChangedFor(nameof(SubmitCommand))]`.
- `IsLoading` (bool) — flaga ładowania, blokuje przycisk Submit podczas trwania żądania HTTP.
- `ErrorMessage` (string) — komunikat błędu wyświetlany w czerwonym TextBlock.
- `SuccessMessage` (string) — komunikat sukcesu (zielony), wyświetlany np. po pomyślnej rejestracji.
- `LoginColor`, `RegisterColor` (string, hex) — aktywna zakładka ma kolor #F97316 (pomarańczowy), nieaktywna #636366 (szary).
- `ButtonMessage` (string) — dynamiczna etykieta głównego przycisku: "Login" lub "Create Account".

Komendy:

- `SwitchToLoginCommand` / `SwitchToRegisterCommand` — synchroniczne komendy przełączające tryb. Ustawiają `IsLoginTab`, kolory zakładek, `ButtonMessage` i czyszczą komunikaty błędów.
- `LoginCommand` — po sukcesie: wywołuje `_sessionService.SetSession(token, username, role, userId)`, następnie `_apiClient.SetAuthToken(token)`, a na końcu

_navigationService.NavigateTo<BrowseViewModel>(). W bloku catch ustawia ErrorMessage.

- RegisterCommand — po sukcesie wywołuje SwitchToLogin() i ClearTextBoxes() w bloku try (nie finally), co przenosi użytkownika z powrotem na zakładkę logowania z wyczyszczonymi polami.
- SubmitCommand — komenda z CanExecute = nameof(CanSubmit). Jest to fasada: jeśli IsLoginTab == true wywołuje LoginAsync(), w przeciwnym razie RegisterAsync().

Logika walidacji CanSubmit(): metoda jest wywoływana automatycznie przez CommunityToolkit za każdym razem, gdy zmieni się którakolwiek z właściwości oznaczonych [NotifyCanExecuteChangedFor(nameof(SubmitCommand))]. Dla trybu logowania: zwraca true jeśli IsValidEmail(Email) || Password.Length != 0 (warunek liberalny — wystarczy poprawny email LUB niepuste hasło). Dla trybu rejestracji: wymaga poprawnego emaila, niepustego Username, niepustego Password oraz Password == PasswordRegister. Przy niezgodności haseł ustawia ErrorMessage na komunikat z PasswordMissMatchException("Passwords doesn't match").Message — jest to eleganckie użycie własnej klasy wyjątku jako nośnika komunikatu, bez faktycznego rzucania wyjątku.

Walidacja emaila używa wyrażenia regularnego `^[^@\s]+@[^\s]+\.[^\s]+\.$` z limitem czasu 250ms, co chroni przed atakami ReDoS.

AuthView.axaml — struktura widoku

Widok renderuje wyśrodkowany panel (Border 420px szerokości) na ciemnym tle #1C1C1E. Struktura:

- Logo — Image + dwa TextBlock ("Movie" białe, "Hub" pomarańczowe).
- Zakładki — dwa przyciski w Grid z ColumnDefinitions="*,*". Kolory tekstu są powiązane z LoginColor i RegisterColor przez Foreground="{Binding LoginColor}".
- Pola formularza — StackPanel z czterema polami. Pole Username i pole PasswordRegister mają IsVisible="{Binding !IsLoginTab}" — widoczne tylko w trybie rejestracji. Pole Password ma PasswordChar="*" dla maskowania.
- Komunikaty — TextBlock z SuccessMessage (zielony) i ErrorMessage (czerwony #FF453A), oba z konwerterem StringConverters.IsNotNullOrEmpty sterującym widocznością.
- Przycisk Submit — Command="{Binding SubmitCommand}", Content="{Binding ButtonMessage}". Przycisk jest automatycznie wyłączony gdy CanSubmit() zwraca false.

8.2. Moduł Browse — przeglądanie katalogu filmów

BrowseViewModel

BrowseViewModel wstrzykuje ApiClient i NavigationService. Odpowiada za pobranie i wyświetlenie listy filmów z możliwością filtrowania po tytule, gatunku i reżyserze.

Inicjalizacja InitializeAsync() używa Task.WhenAll(LoadGenresAsync(), LoadDirectorsAsync()) — oba żądania HTTP wykonują się równolegle, co skraca czas inicjalizacji. Po ich zakończeniu wywołuje LoadMoviesAsync().

Kolekcje obserwowalne:

- Movies (ObservableCollection<MovieSummaryViewModel>) — główna kolekcja kart filmów wyświetlanych w siatce.

- **Genres** (`ObservableCollection<GenreFilterItem>`) — lista gatunków do `ComboBox` filtra. Pierwszy element to zawsze `GenreFilterItem(null, "All Genres")`.
- **Directors** (`ObservableCollection<DirectorFilterItem>`) — lista reżyserów do `ComboBox` filtra. Pierwszy element to `DirectorFilterItem(null, "All Directors")`.

Właściwości obserwowalne:

- **SearchTitle** (`string`) — tekst wyszukiwania po tytule, powiązany z `TextBox`.
- **SelectedGenreId** (`string`) — ID wybranego gatunku jako `string`. `ComboBox` używa `SelectedValueBinding="{Binding IdString}"` — mechanizm Avalonia dopasowuje wybrany element przez wartość `IdString`, a nie przez referencję obiektu.
- **SelectedDirectorId** (`string`) — analogicznie dla reżysera.
- **IsLoading** (`bool`) — steruje widocznością `ProgressBar`.

Rekordy pomocnicze `GenreFilterItem` i `DirectorFilterItem` definiują właściwość `IdString => Id?.ToString() ?? string.Empty`, która jest kluczowa dla mechanizmu `SelectedValueBinding` w Avalonia — `ComboBox` porównuje `SelectedValue` (`string`) z `IdString` każdego elementu, aby ustalić zaznaczony element.

Komenda `SearchCommand` wywołuje `LoadMoviesAsync()` — powiązana zarówno z przyciskiem wyszukiwania, jak i z `KeyBinding Gesture="Return"` na `TextBox`. Zmiana wyboru w `ComboBox` natychmiast aktualizuje `SelectedGenreId/SelectedDirectorId` w `ViewModel` przez dwukierunkowy binding, jednak wyszukiwanie jest wyzwalane dopiero przez `SearchCommand` (Enter lub przycisk) — nie ma automatycznego wyszukiwania przy każdej zmianie filtra.

MovieSummaryViewModel

Lekki `ViewModel`-wrapper dla `MovieSummaryResponse`. Przechowuje niezmiennie właściwości: `Id`, `Title`, `Year`, `GenreName`, `PosterUrl`, `AverageRating`. Właściwość obliczana `ScoreDisplay` formatuje ocenę jako "7.3" lub "—" gdy brak ocen. Komenda `SelectCommand` wywołuje przekazany callback `onSelected(Id)`, co inicjuje nawigację do szczegółów.

BrowseView.axaml — struktura widoku

Widok zbudowany jako `DockPanel` z ciemnym tłem `#141414`. Nagłówek z filtrami (`DockPanel.Dock="Top"`) — `Grid` z trzema kolumnami: `TextBox` z `Text="{Binding SearchTitle}"` i `KeyBinding` na Enter → `SearchCommand`; `ComboBox` gatunków z `ItemsSource="{Binding Genres}"`, `SelectedValue="{Binding SelectedGenreId}"`; `ComboBox` reżyserów analogicznie. `ProgressBar` — `IsIndeterminate="True"`, pomarańczowy pasek 3px wysokości. Siatka filmów — `ScrollViewer` zawierający `ItemsControl` z `WrapPanel Orientation="Horizontal"`. Każdy film renderowany jest jako karta 200px szerokości z: plakatem (`Image Source="{Binding PosterUrl}"`) lub emoji 🎬 gdy brak URL; tytułem (`TextWrapping="Wrap"`, `MaxLines="2"`); rokiem i gatunkiem (badge z tłem `#3A2A00` i tekstem `#F5A623`); oceną (gwiazdka ★ + `ScoreDisplay` + "/ 10"). Cała karta jest opakowana w `Button` z `Command="{Binding SelectCommand}"` i `Cursor="Hand"`.

8.3. Moduł MovieDetail — szczegóły i ocenianie

MovieDetailViewModel

Najbardziej rozbudowany `ViewModel` w projekcie. Wstrzykuje `ApiClient` i `SessionService`.

MovieId (int) — kluczowa właściwość. ViewModel nadpisuje metodę częściową `partial void OnMovieIdChanged(int value)`, która natychmiast wywołuje `LoadMovieCommand.ExecuteAsync(null)`. Dzięki temu przypisanie `detailVm.MovieId = movieId` w `NavigationService` automatycznie wyzwała ładowanie danych.

Właściwości obliczane (bez `[ObservableProperty]`):

- **DirectorFullName** — `($"{Director.FirstName} {Director.LastName}")`. Po załadowaniu danych ViewModel ręcznie wywołuje `OnPropertyChanged(nameof(DirectorFullName))`, bo właściwość obliczana nie ma automatycznego powiadamiania.
- **GenreName** — `Genre?.Name ?? string.Empty`, analogicznie.
- **ScoreDisplay** — `AverageRating.Value.ToString("F1")` lub `"Not rated yet"`.

Właściwości obserwowalne dotyczące oceniania:

- **UserRating (int?)** — zmiana tej właściwości wyzwała `partial void OnUserRatingChanged(int? value)`, która iteruje po kolekcji `Stars` i ustawia `IsFilled = value.HasValue && star.Value <= value.Value` — wizualnie wypełnia gwiazdki do wybranej wartości.
- **ShowCommentBox (bool)** — steruje widocznością pola komentarza i przycisku `Submit`.
- **Comment (string)** — treść komentarza.
- **ExistingRatingId (int?)** — ID istniejącej oceny użytkownika (jeśli już oceniał). Decyduje o tym, czy `SubmitRatingAsync` wywoła `POST` (nowa ocena) czy `PUT` (aktualizacja).

Kolekcja `Stars`: `ObservableCollection<StarViewModel>` z 10 elementami (wartości 1–10). Tworzona jednorazowo w `LoadMovieAsync()` (flaga `_starsAdded` zapobiega duplikacji przy ponownym ładowaniu). Każdy `StarViewModel` przechowuje `Value (int)` i `IsFilled (bool, [ObservableProperty])`. Komenda `SelectCommand` wywołuje callback `SetRating(value)` w `MovieDetailViewModel`, który ustawia `UserRating = value` i `ShowCommentBox = true`.

`SubmitRatingAsync` sprawdza `ExistingRatingId`: jeśli `null` wywołuje `CreateRatingAsync` (`POST /api/movies/{id}/ratings`), jeśli ma wartość wywołuje `UpdateRatingAsync` (`PUT /api/ratings/{id}`). Po sukcesie: `ShowCommentBox = false`, a następnie ponowne wywołanie `LoadMovieAsync()` odświeża dane (nowa średnia, zaktualizowana lista ocen). `LoadMovieAsync()` szuka w liście `Ratings` oceny, której `UserId == session.UserId` — jeśli znajdzie, ustawia `UserRating` i `ExistingRatingId`, co wizualnie zaznacza gwiazdki poprzedniej oceny użytkownika.

MovieDetailView.axaml — struktura widoku

Widok podzielony na dwie kolumny (`Grid.ColumnDefinitions="Auto,*"`). Lewa kolumna (320px) — plakat w `Border` z `CornerRadius="12"` i `ClipToBounds="True"`. Jeśli `PosterUrl` jest pusty, wyświetla emoji 🎬 (64px). Prawa kolumna — `ScrollViewer` z `StackPanel`:

- **Tytuł** — `TextBlock` 36px, pogrubiony, biały, z zawijaniem tekstu.
- **Metadane** — rok (szary), badge gatunku (pomarańczowy na ciemnym tle), ocena (★ + `ScoreDisplay` + `" / 10"`).
- **Reżyser** — etykieta "DIRECTOR" (szara, spaced) + `DirectorFullName`.

- Gwiazdki oceny — `ItemsControl` z `StackPanel Orientation="Horizontal"`. Każda gwiazdka to `Button 28×28px`. Wewnątrz `Panel` dwa obrazy `star.png` — jeden z `Opacity="0.2"` (pusta gwiazdka, `IsVisible="{Binding !IsFilled}"`), drugi pełny (`IsVisible="{Binding IsFilled}"`). Mechanizm przełączania widoczności zamiast zmiany koloru jest prostym i skutecznym rozwiązaniem dla Avalonia.
- Pole komentarza — `StackPanel IsVisible="{Binding ShowCommentBox}"` z `TextBox` (wieloliniowy, `MaxHeight="100"`) i przyciskiem "Submit rating" → `SubmitRatingCommand`.
- Opis — `TextBlock` z `LineHeight="22"` dla czytelności.
- Błąd — czerwony `TextBlock` z `ErrorMessage`.

8.4. Moduł Admin — panel zarządzania katalogiem

Architektura panelu Admin

Panel Admin jest zbudowany w architekturze master-detail z dynamicznym formularzem. `AdminViewModel` pełni rolę routera — przechowuje właściwość `ActiveForm` (typ `object?`), do której przypisuje instancje różnych sub-`ViewModel`ów. `AdminView.axaml` używa `ContentControl` z wieloma `DataTemplate` — Avalonia automatycznie dobiera właściwy szablon na podstawie typu runtime obiektu w `ActiveForm`. Jest to wzorec `implicit data template`, który eliminuje potrzebę ręcznego przełączania widoczności paneli.

Dostęp do panelu Admin jest kontrolowany na poziomie UI przez `MainWindowViewModel.IsAdmin`, które deleguje do `SessionService.IsAdmin` (sprawdza `Role == "Admin"`). W `MainWindowView.axaml` przycisk nawigacji do panelu ma `IsVisible="{Binding IsAdmin}"` — użytkownik bez roli Admin nie widzi przycisku. Jest to autoryzacja po stronie klienta (UI-level), uzupełniona autoryzacją po stronie serwera (API wymaga tokenu z rolą Admin dla operacji CRUD).

AdminViewModel i sub-ViewModele formularzy

`AdminViewModel` wystawia pięć komend nawigacyjnych: `ShowAddMovieCommand`, `ShowEditMovieCommand`, `ShowDeleteMovieCommand`, `ShowAddGenreCommand`, `ShowAddDirectorCommand`. Każde przypisanie do `ActiveForm` wyzwała `partial void OnActiveFormChanged(object? value)`, która emituje event `ActiveFormChanged`. Event ten jest subskrybowany w code-behind `AdminView.axaml.cs` — po zmianie formularza wywołuje `InitializeAsync()` na nowym sub-`ViewModel`ie (jeśli go implementuje), co ładuje dane potrzebne formularzowi (listy filmów, gatunków, reżyserów).

`AddMovieFormViewModel` ładuje listy gatunków i reżyserów równolegle w `InitializeAsync()`. `SubmitCommand` waliduje niepustość tytułu i wybór gatunku i reżysera. Wywołuje `api.CreateMovieAsync(new CreateMovieRequest { ... })`. Po sukcesie ustawia `SuccessMessage` i czyści pola tytułu, opisu i URL plakatu (rok i wybory `ComboBox` pozostają dla wygody przy dodawaniu kolejnych filmów).

`EditMovieFormViewModel` zawiera dodatkową właściwość `SelectedMovie` (`MovieListItem?`) — `ComboBox` z listą wszystkich filmów. `Partial void OnSelectedMovieChanged(MovieListItem? value)` wywołuje `LoadMovieDetails(value.Id)`, która pobiera pełne dane wybranego filmu z API i wypełnia pola formularza. Właściwość `MovieLoaded` (`bool`) steruje widocznością `StackPanel` z polami edycji — są ukryte dopóki nie wybrano filmu.

`DeleteMovieFormViewModel` po usunięciu usuwa film z lokalnej kolekcji `Movies.Remove(SelectedMovie)` i zeruje `SelectedMovie = null` — UI natychmiast odzwierciedla usunięcie bez ponownego ładowania z API.

AddGenreFormViewModel — minimalny formularz z jednym polem Name.

AddDirectorFormViewModel — formularz z polami FirstName, LastName (wymagane) i Nationality (opcjonalne). Oba po sukcesie czyszczą pola i wyświetlają SuccessMessage.

AdminView.axaml — struktura widoku

Widok podzielony na dwie kolumny (Grid ColumnDefinitions="220,*"). Lewa kolumna — menu boczne (220px): Border z ciemnym tłem #1C1C1E i prawą krawędzią #2C2C2E. Zawiera StackPanel z przyciskami pogrupowanymi pod nagłówkami "MOVIES", "GENRES", "DIRECTORS". Przycisk "Delete Movie" ma Foreground="#FF453A" (czerwony) — wizualne ostrzeżenie o destruktywnej operacji. Prawa kolumna — ScrollView zawierający ContentControl Content="{Binding ActiveForm}". Wewnątrz ContentControl.DataTemplates zdefiniowanych jest pięć DataTemplate, każdy z DataTypeem wskazującym na konkretny typ sub-ViewModelu. Każdy formularz ma maksymalną szerokość 500px i margines 32px.

8.5. Podsumowanie przepływu danych

Poniżej schematyczny przepływ dla typowej operacji — kliknięcia karty filmu w widoku Browse:

1. Użytkownik klika kartę → Button.Command="{Binding SelectCommand}" → MovieSummaryViewModel.Select() → callback NavigateToMovie(id) w BrowseViewModel.
2. BrowseViewModel wywołuje _navigation.NavigateToMovie(id).
3. NavigationService tworzy nowy MovieDetailViewModel przez fabrykę DI, ustawia CurrentView = detailVm, następnie detailVm.MovieId = id.
4. Setter MovieId (wygenerowany przez CommunityToolkit) wywołuje OnPropertyChanged("MovieId") i partial void OnMovieIdChanged(int value).
5. OnMovieIdChanged wywołuje LoadMovieCommand.ExecuteAsync(null).
6. LoadMovieAsync() pobiera dane z API, wypełnia właściwości, wywołuje OnPropertyChanged dla właściwości obliczanych.
7. NavigationService.CurrentView zmienia się → PropertyChanged → MainWindowViewModel.OnPropertyChanged("CurrentView") → Avalonia odświeża ContentControl → ViewLocator dobiera MovieDetailView → widok renderuje się z danymi z ViewModelu.

8.6. Kluczowe właściwości architektoniczne

- Separacja odpowiedzialności — każda klasa posiada jednoznacznie zdefiniowany zakres odpowiedzialności, zgodny z zasadą pojedynczej odpowiedzialności (Single Responsibility Principle).
- Odwrócenie zależności — komponenty wyższego poziomu (modele widoków) zależą od abstrakcji (interfejsów serwisów), a nie od konkretnych implementacji.
- Reaktywność — zmiany stanu propagowane są przez mechanizm INotifyPropertyChanged, eliminując konieczność ręcznego odświeżania interfejsu.
- Konwencja ponad konfiguracją — ViewLocator eliminuje potrzebę jawnej rejestracji par ViewModel–View, redukując ilość kodu konfiguracyjnego.
- Testowalność — wstrzykiwanie zależności przez konstruktor umożliwia podstawianie atrap (mocks) w testach jednostkowych modeli widoków.

9. Wdrożenie produkcyjne — Azure

9.1. Zasoby Azure

Zasób	Konfiguracja
App Service	moviehub-api · Free F1 · Linux · .NET 10 · Poland Central
PostgreSQL Flexible Server	moviehub-db · Burstable B1ms · PostgreSQL 16 · Poland Central
Resource Group	moviehub-rg · Poland Central
URL API	https://moviehub-api.azurewebsites.net
Scalar UI (produkcja)	https://moviehub-api.azurewebsites.net/scalar/v1

9.2. CI/CD — GitHub Actions

Wdrożenie jest wyzwalane automatycznie przez push do gałęzi master. Uwierzytelnianie odbywa się poprzez OIDC (User-assigned managed identity) — nie są przechowywane żadne sekrety w repozytorium. Workflow: `.github/workflows/master_moviehub-api.yml`.

Harmonogram zatrzymywania i uruchamiania bazy danych jest obsługiwany przez dodatkowy workflow (`db-schedule.yml`): automatyczne wyłączenie o 22:00 i uruchomienie o 06:00 czasu polskiego — pozwala to uniknąć naliczania kosztów poza godzinami pracy.

9.3. Zmienne środowiskowe (App Service)

Klucz	Wartość / opis
<code>ConnectionStrings__DefaultConnection</code>	Connection string do Azure PostgreSQL (SslMode=Require)
<code>Jwt__SecretKey</code>	Klucz symetryczny do podpisywania tokenów JWT
<code>Jwt__Issuer</code>	moviehub-api
<code>Jwt__Audience</code>	moviehub-client
<code>Jwt__ExpirationMinutes</code>	60
<code>ASPNETCORE_ENVIRONMENT</code>	Production

9.4. Dostęp do bazy danych i zarządzanie

Migracje EF Core są aplikowane ręcznie z poziomu terminala z użyciem connection stringa produkcyjnego:

```
cd src/MovieHub.API
```

```
dotnet ef database update --connection
"Host=moviehub-db.postgres.database.azure.com;
  Port=5432;Database=moviehub;Username=moviehub;Password=...;SslMode=Require"
```

Przyznanie roli Admin odbywa się bezpośrednio przez SQL:

```
UPDATE "Users" SET "Role" = 1 WHERE "Email" = 'user@example.com';
```

10. Struktura projektu

Projekt MovieHub jest zorganizowany jako monorepozytorium zawierające dwa główne projekty .NET: MovieHub.API (backend) oraz MovieHub.Client (frontend).

10.1. Struktura katalogów

```
MovieHub/
├─ MovieHub.slnx
├─ docker-compose.yml
├─ .github/workflows/
│   ├─ master_moviehub-api.yml    (CI/CD – deploy on push to master)
│   └─ db-schedule.yml            (auto stop/start bazy danych)
└─ src/
    ├─ MovieHub.API/
    │   ├─ Controllers/           (Auth, Genres, Movies, Ratings, Directors)
    │   ├─ Services/              (Auth, Genre, Movie, Rating, Director)
    │   ├─ Models/                (User, Genre, Director, Movie, Rating, UserRole)
    │   ├─ DTOs/                  (Auth/, Movies/, Ratings/, Genres/, Directors/)
    │   ├─ Exceptions/            (ConflictException, NotFoundException,
    │   │                          ForbiddenException, DuplicateUserException,
    │   │                          InvalidCredentialsException)
    │   ├─ Options/                (JwtSettings)
    │   └─ Data/                  (AppDbContext.cs)
    │   └─ Program.cs
    └─ MovieHub.Client/
        ├─ Views/                 (MainWindowView, AuthView, BrowseView,
        │                          MovieDetailView, AdminView)
        └─ ViewModels/            (MainWindowViewModel, AuthViewModel,
```

	BrowseViewModel, MovieDetailViewModel, MovieSummaryViewModel, StarViewModel)
Admin/	(AddMovieFormViewModel, EditMovieFormViewModel, DeleteMovieFormViewModel, AddGenreFormViewModel, AddDirectorFormViewModel)
Services/	(ApiClient, NavigationService, SessionService)
Models/	(DTOs mirrorujące odpowiedzi API)
Exceptions/	(PasswordMissMatchException)
Assets/	(logo.png, star.png, admin.png, logout.png)
App.axaml	(kontener DI)

10.2. Konwencje nazewnnicze i organizacja kodu

Projekt stosuje jednolite konwencje nazewnicze obowiązujące w całym kodzie. W warstwie backendowej kontrolery są pogrupowane według zasobu domenowego, a DTOs są organizowane w podfolderach odpowiadających obszarom funkcjonalnym (Auth/, Movies/, Ratings/, Genres/, Directors/). W warstwie frontendowej obowiązuje konwencja ViewLocator: klasa o nazwie XViewModel jest automatycznie wiązana z widokiem XView poprzez podmianę fragmentu "ViewModel" na "View" w przestrzeni nazw, co eliminuje potrzebę ręcznej rejestracji par widok–ViewModel.

11. Podsumowanie

Projekt MovieHub stanowi kompletny, wdrożony system informatyczny klasy full-stack, zrealizowany samodzielnie w ramach projektu zaliczeniowego. Aplikacja implementuje pełny cykl życia oprogramowania — od zaprojektowania modelu danych i architektury API, przez implementację logiki biznesowej i interfejsu użytkownika, aż po wdrożenie produkcyjne w chmurze Azure z automatycznym pipeline'em CI/CD.

W trakcie realizacji projektu zastosowano szereg wzorców i technik inżynierii oprogramowania: architekturę warstwową (kontroler → serwis → DbContext), wzorzec MVVM po stronie frontentu, JWT jako mechanizm bezstanowego uwierzytelniania, wzorzec DTO separujący kontrakt API od modelu domenowego, a także hierarchię własnych wyjątków domenowych mapowanych na kody HTTP. Dbłość o bezpieczeństwo przejawia się m.in. w haszowaniu haseł algorytmem BCrypt (koszt 12), ochronie przed atakami czasowymi w procesie logowania oraz separacji autoryzacji na poziomie UI i API.

11.1. Zrealizowane wymagania funkcjonalne

- Rejestracja i logowanie użytkowników z uwierzytelnianiem JWT
- Przeglądanie katalogu filmów z filtrowaniem po tytule, gatunku i reżyserze
- Widok szczegółów filmu z listą ocen i obliczoną średnią
- Widget oceniania gwiazdkowego (10 gwiazdek) z możliwością dodania komentarza oraz edycji wystawionej oceny
- Panel administracyjny: dodawanie, edycja i usuwanie filmów, dodawanie gatunków i reżyserów
- Kontrola dostępu oparta na rolach (RBAC): użytkownik / administrator

- Wdrożenie produkcyjne w chmurze Azure z automatycznym CI/CD przez GitHub Actions

11.2. Zastosowane technologie — zestawienie

Obszar	Technologia	Wersja / Uwagi
Backend	ASP.NET Core Web API	.NET 10/
ORM	Entity Framework Core	Code-First, Fluent API, migracje
Baza danych	PostgreSQL	v16, Docker lokalnie / Azure produkcja
Frontend	Avalonia UI (MVVM)	CommunityToolkit.Mvvm, cross-platform
Autentykacja	JWT + BCrypt	HMAC-SHA256, koszt 12
Walidacja	FluentValidation	Warstwa API
Chmura / CI/CD	Microsoft Azure + GitHub Actions	OIDC, App Service F1, Poland Central
Środowisko dev	Arch Linux, JetBrains Rider	Educational Pack, Docker